🏗️ Infraestructura como código

IA como Copiloto · Parte 7 de 8

🌱 Nivel: avanzado · ☁️ Terraform · Docker Compose · Containerlab · Ansible

Sergi Garcia Barea · Octubre 2026 · CC BY-SA 4.0

💬 La infraestructura también es código

✅ Antes (a mano)

- ✅ Servidor configurado a pulso, nobody sabe exactamente qué
- ✅ "Funciona en mi máquina" también en servidores
- ✅ Recrear el entorno idéntico: días

🎯 Ahora (IaC + copiloto)

- ✅ La infra vive en ficheros versionados en el repo
- ✅ El agente la genera, la revisa y la despliega contigo
- ✅ Recrear el entorno idéntico: minutos

💡 Si tu infraestructura está en un documento… no está descrita: está *recordada*. El código se ejecuta; el documento se interpreta.

🔊

🗺️ El mapa IaC: qué herramienta para qué

Capa	Herramienta	Le describe al copiloto…	Tip de repo
Contenedores	Docker + Compose	Apps y servicios, redes y volúmenes del equipo dev o demo	`compose.yaml` en la raíz
Servidores	Ansible	Configuración declarativa de máquinas (playbooks idempotentes)	`ansible/`
Nube	Terraform	Recursos: VMs, redes, DNS, colas. Plan antes de apply	`infra/` o repo aparte
Redes de lab	Containerlab	Topologías de red con routers/switches reales (vrnetlab, cEOS…)	`clab/` + topología YAML
Kubernetes	Helm / K8s	Despliegue en clúster, portando el compose a charts a charts	`k8s/` o charts

🎯 El copiloto conoce todas: pídele "convierte el compose a un chart básico de Helm" y lo hace.

🔊

🐳 Ejemplo 1 · Docker Compose con criterio

💬 PROMPT

```
"Genera el compose.yaml de la demo:
- web (FastAPI, puerto 8000) y db
  (postgres:16, salud y retry)
- volumen persistente para datos
- red interna, sin puertos para db
- healthchecks en ambos servicios
- .env para las contraseñas
Versiona y explica por qué."
```

OpenCode · Compose

✅ LO QUE TE PIDE EL DETALLE

- ✅ **Healthcheck** en db: web no arranca antes de que esté lista
- ✅ **depends_on con condition**: orden real de arranque
- ✅ **.env fuera del compose**: nada de secretos versionados
- ✅ **db sin puertos publicados**: solo la red interna
- ✅ El agente **verifica**: levanta, prueba y baja el stack

🔊

☁️ Ejemplo 2 · Terraform: plan antes de apply

💬 PROMPT

```
"En infra/: Terraform que crea
- una VPC con 2 subredes públicas
- una VM Ubuntu + reglas SG (22, 443)
- etiquetas name=proyecto-demo
State remoto que declaro en
backend.tf. PRIMERO plan y muéstra
las diferencias; nada de apply
sin mi listo."
```

Freebuff OpenCode · Terraform

✅ DETALLES QUE IMPORTAN

- ✅ **plan antes que apply**: el plan es el diff que revisas
- ✅ **State remoto** (S3/GCS): con lock, nada de state local en tu portátil
- ✅ **Variables en variables.tf**, valores en terraform.tfvars (gitignored)
- ✅ Un recurso nuevo por vez; con el agente, **plan pequeño**
- ✅ Para practicar gratis: **LocalStack** o cuenta sandbox sin tarjeta

🔊

🧪 Ejemplo 3 · Containerlab: topología de red

💬 PROMPT

```
"Genera clab/demo.yml: 2 routers
(ceos) + 2 hosts. R1 conecta con
host1 y R2 con host2; link
R1-R2 en 10.0.0.0/30.
Encapsula la topología con:
- Addressing (loopbacks, mgmt)
- Arranque con clab deploy y
  pruebas: ping host1-host2"
```

OpenCode · Containerlab

✅ IDEAL PARA DOCENCIA

- ✅ **Teaching labs**: cada alumno despliega la misma topología en 2 min
- ✅ El agente puede **verificar**: ping entre nodos y devuelve capturas del output
- ✅ Topología en **YAML**: diff en PR igual que el resto del repo
- ✅ Alternativa ligera a laboratorios físicos o GNS3 pesado

🔊

⚙️ Ejemplo 4 · Ansible: playbooks idempotentes

💬 PROMPT

```
"En ansible/: playbook que
configura los servidores web:
- actualizar paquetes base
- instalar nginx + copiar conf
- abrir 80/443 en el firewall
- service nginx habilitado
Idempotente: ejecutarlo dos veces
no cambia nada la segunda."
```

OpenCode · Ansible

✅ QUÉ VIGILAR

- ✅ **Idempotencia**: el playbook se puede repetir sin efectos secundarios
- ✅ **Inventario** separado del playbook (ini/yaml, en `ansible/inventory/`)
- ✅ Secretos con **ansible-vault**, nunca en claro
- ✅ Pide al agente que **ejecute con --check** antes del real (dry-run)

🔊

🔄 El flujo IaC con copiloto

1. **Declara el estado deseado** — "la demo levanta con web + db y datos persistentes"
2. **El agente genera o propone el cambio** — ficheros Terraform/Compose/Ansible
3. **PLAN/DIFF en la infra** — `terraform plan`, `compose config`, `ansible --check`: revisa el diff, tu compuerta de seguridad
4. **Apply bajo tu supervisión** — en local/demo primero; producción fuera solo con doble revisión
5. **Versiona** — commit de ficheros, no de "cambios manuales" en la consola

🥅 Criterios de aceptación para la IaC: `terraform plan` limpio, `compose up` pasa healthchecks y `ansible --check` sin sorpresas.

🔊

🛡️ Riesgos reales de la IaC con IA

1. El "apply decidido por la IA"	← Nunca: el agente propone, tú ejecutas (o apruebas en el pipeline)
2. Secretos en ficheros	← .env, tfvars, vault. y SIEMPRE gitignore + escaneo en CI
3. Drift: la consola manda	← Si alguien toca a mano fuera del código, IaC miente. Prohibido o detectado (tfsec/drift detection)
4. Alucinaciones de API	← El agente puede inventar atributos: verifica contra la docs (MCP) y valida con `terraform validate`
5. Costes por accidente	← Plan grande = factura grande. Pide sandbox/cuenta demo y alertas de coste

🎯 Regla del proyecto: **nada que cueste dinero se ejecuta sin diff + humano**. La IA escribe infra; tu equipo la aprueba.

🔊

🎓 Reto del curso

🧪 Mini-reto IaC

Diseña con el copiloto un lab: compose con app + BD para tu asignatura, y Containerlab con una topología de 2 routers

🤖 Prompt de arranque

```
"Convierte la descripción de mi
demo en infra-as-code: compose +
healthchecks + .env de ejemplo.
Primero propón el plan, no
escribas ficheros aún."
```

Material CC BY-SA · Más ejemplos y prompts completos en el repo

🔊

✨ En esta parte 7



IaC = infra versionada

Compose · Ansible · Terraform · Containerlab: describir el estado deseado y revisar el diff



El copiloto propone

La IA genera y verifica, pero el apply lo aprueba un humano; secretos y costes siempre bajo control

Última: **Documentación como código** — diagramas y docs que evolucionan con el código. 🚀

CC BY-SA 4.0 · Sergi Garcia Barea · Octubre 2026